

Can Retrieval-Augmented Few-Shot Prompting Match LoRA Fine-Tuning for Issue Report Classification? An Empirical Study

Anonymous author

Anonymous affiliation

Abstract

Background. Issue report classification (IRC) accelerates modern software maintenance, yet it remains a challenging task. State-of-the-art automated IRC methods fine-tune Large Language Models (LLMs) with Low-Rank Adaptation (LoRA), which is computationally expensive and must be repeated to incorporate newly labeled issue reports or whenever the model is swapped.

Aims. We empirically investigate the hypothesis that low-cost, retrieval-augmented generation (RAG) based approaches can achieve competitive IRC performance with the LoRA fine-tuning baseline. If validated, this would introduce the possibility of cost-efficient IRC solutions; An avenue that, to our knowledge, remains unexplored in the literature.

Method. To test this hypothesis, we conduct a case study on the Qwen2.5-Instruct family, a set of open-source models with strong public-benchmark performance, evaluated at four parameter sizes to analyze the effect of model scale. We propose three retrieval-based methods — VOTAG, RAGTAG, and BRAGTAG — each of which works by retrieving the top- k most similar issues from an index of labeled issues and reasoning on them. Specifically, VOTAG assigns the label via a similarity-weighted vote over the retrieved neighbors; RAGTAG presents the neighbors as classified examples in a few-shot LLM prompt; and BRAGTAG extends RAGTAG with a more balanced retrieval scheme that better covers the label space. We evaluate all three methods against a LoRA fine-tuning baseline on an 11-project benchmark, under both *project-specific* and *project-agnostic* data scope settings.

Results. With project-specific data, RAGTAG outperforms fine-tuning by 0.021–0.040 macro F_1 across all model sizes. Fine-tuning on cross-project data beats project-specific RAGTAG by only 0.029 aggregate macro F_1 despite $11\times$ more labeled data. The balanced retrieval approach, BRAGTAG closes this performance gap, achieving statistical equivalence with fine-tuning when using a retrieval fallback for invalid LLM outputs. Averaged across model sizes, retrieval-augmented few-shot prompting requires 33% less peak GPU memory than fine-tuning.

Conclusions. Under our experimental setup and case study, RAG-based approaches demonstrate competitive performance against LoRA fine-tuning, suggesting they represent a viable and cost-efficient alternative for IRC, one that the literature has yet to fully explore.

2012 ACM Subject Classification Software and its engineering → Software notations and tools; Computing methodologies → Natural language processing

Keywords and phrases Issue classification, Retrieval-augmented generation, Fine-tuning, Large language models, Mining software repositories

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Issue Tracking Systems provide a structured way to communicate concerns related to the software project by allowing users to submit issue reports [14]. Issue reports can be classified with labels to help with triage. Effective issue report classification (IRC) results in faster issue resolution [35], and is crucial for many software maintenance tasks, such as issue



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

prioritization [10], issue assignment [49], and developer onboarding [42, 6]. However, studies show that labels are rarely applied in issue creation [4, 24, 30]. As a result, project maintainers must manually classify the issues for triage, which is laborious and time-consuming [19].

To address this problem, numerous automated methods have been proposed around supervised learning over three established issue report categories: *bug*, *feature*, and *question* [4, 30, 28, 6, 5]. Early work relied on classic machine learning (ML) techniques [4, 30, 19, 24], while more recent studies fine-tune transformer models on labeled issue reports [28, 29, 44, 8, 11, 47, 22]. These approaches require a substantial amount of training data and often struggle with minority classes [28, 29, 11, 23, 45]. The current state-of-the-art methods fine-tune Large Language Models (LLMs) using Low-Rank Adaptation (LoRA [25]) [23, 6, 5]. However, fine-tuning LLMs is computationally expensive and must be repeated whenever the model is swapped or to incorporate new labeled issues that accumulate in the project [19, 30].

Prior work shows that placing a few demonstration examples in the prompt can boost an LLM’s performance on specialized tasks [7, 34, 39, 27, 9] through in-context learning [9]. Few-shot performance, however, depends on the choice of in-context examples. Studies show that using the query’s retrieved nearest neighbors as in-context examples yields strong few-shot performance [36, 51]. Additionally, the nearest neighbors tend to share the same label as the query [15], making them suitable in-context examples. The combination of these insights leads to retrieval-augmented few-shot prompting, where the classified issue reports most similar to a query are retrieved and used as its in-context examples. Since this method enhances performance without model training or weight updates, it has the potential to serve as a cost-efficient alternative to LoRA fine-tuning for IRC. Yet both a detailed empirical evaluation of this approach for IRC and its comparison against fine-tuning remain underexplored. To address these gaps, we conduct an empirical study that examines whether it is possible for retrieval-augmented few-shot to achieve IRC performance competitive with LoRA fine-tuning in practice.

An important factor in retrieval-augmented few-shot performance is the number of provided examples: too few may not provide enough context, and too many can add noise [36]. Since the in-context examples are the query’s nearest neighbors, they can classify the query under the standard k -nearest-neighbor (KNN) framework [15, 18]. Therefore, the point where adding neighbors stops improving the retrieval-only classification informs an upper bound for the number of in-context examples. Additionally, retrieval-only IRC provides a non-parametric baseline that isolates the LLM’s contribution from that of retrieval in retrieval-augmented few-shot prompting. To our knowledge, no prior study has explored such a baseline for IRC. Therefore, we pose our first research question:

RQ1. How effectively can retrieval alone classify issue reports, and at what point does adding more retrieved neighbors stop helping? We answer this question by applying a standard distance-weighted KNN classification [15, 18]. Given a query issue, its top- k most similar issue reports (nearest neighbors) are first retrieved from an embedded index of existing issue reports. Next, the label is predicted with a similarity-weighted vote over the neighbors. We evaluate this method over a wide range of top- k values to measure its peak performance and identify the point where additional neighbors are not beneficial (best k value). With the best k value established, we ask:

RQ2. How well does retrieval-augmented few-shot prompting classify issue reports, and how does performance vary with the number of in-context examples? We answer this question by presenting the same top- k nearest issue reports as classified examples in a few-shot prompt, instructing the model to predict a label for the query issue. $k=0$ results in a zero-shot LLM baseline similar to prior work [12]. We evaluate this approach with

different model scales over a range of k values bounded by RQ1. RQ2 evaluations reveal that retrieval-augmented few-shot has the weakest performance on the question class, and question issues are frequently misclassified as bugs. Zero-shot prompting results indicate that the models are already biased towards question-to-bug misclassifications even before the examples are presented. We hypothesize that bug-labeled neighbors retrieved for true-question queries reinforce this misclassification. This motivates our next research question:

RQ3. Can a balanced retrieval intervention reduce question-to-bug misclassification and improve retrieval-augmented few-shot performance?

We answer this question by exploring a more balanced retrieval technique that removes bug-labeled neighbors from the prompt whenever the retrieved neighbors' label distribution suggests the query is likely a question. We evaluate it over the same k values and model scales used for RQ2. For brevity, we refer to the retrieval-only classifier (RQ1) as **V**oting-based **T**AGging (**VOTAG**), the retrieval-augmented few-shot approach (RQ2) as **R**etrieval-**A**ugmented **G**enerative **T**AGging (**RAGTAG**), and its balanced variant (RQ3) as **B**alanced **R**AGTAG (**BRAGTAG**). We compare these methods against the established LoRA [25] fine-tuning baseline [23, 6, 5] to answer our final research question:

RQ4. How do retrieval-based methods compare against LoRA fine-tuning for IRC in classification performance and computational cost?

We conduct this comparison using the Qwen2.5-Instruct model family at 3B, 7B, 14B, and 32B parameters [50] to analyze the effect of model scale in our evaluations. We use Heo et al.'s [23] 11-project dataset for our experiments and adopt its two evaluation settings: a *project-specific* (PS) configuration where retrieval/training is over each project's individual data split in isolation, and a *project-agnostic* (PA) configuration where retrieval/training is over data from the 11 projects pooled together.

Results show that retrieved few-shot examples consistently improve IRC performance. Even a single retrieved example outperforms the zero-shot baseline across all model sizes (Section 5.2). LLM-based reasoning also consistently improves over pure retrieval: every RAGTAG configuration outperforms VOTAG's best result. Notably, however, VOTAG comes within 0.018 macro F_1 of Qwen-3B zero-shot performance (Section 5.1). Retrieval-augmented few-shot prompting is also competitive with LoRA fine-tuning. Using only each project's own labeled issue reports under the PS data scope, RAGTAG outperforms project-specific LoRA fine-tuning by 0.021–0.040 macro F_1 across all model sizes (Section 5.4). LoRA fine-tuning only catches up when trained under the PA data scope, which pools $11\times$ more labeled data across projects. Even then, RAGTAG remains competitive: aggregated across all model sizes, project-agnostic LoRA fine-tuning leads project-specific RAGTAG by only 0.029 macro F_1 , and the two methods are statistically tied at Qwen-3B and Qwen-32B. BRAGTAG further narrows this gap by reducing question-to-bug misclassifications without sacrificing performance on the other labels (Section 5.3). The aggregate macro F_1 difference between BRAGTAG and LoRA fine-tuning is -0.010 ($\text{BRAGTAG} - \text{LoRA fine-tuning}$). Applying a VOTAG fallback to invalid LLM outputs across all methods further tightens the comparison to statistical equivalence, passing *two one-sided tests (TOST)* [33] at $\delta=0.01$. Retrieval-augmented few-shot offers deployment advantages as well. Averaged across the four model sizes, RAGTAG and BRAGTAG require 33% less peak GPU memory than fine-tuning (Section 5.4). Additionally, incorporating newly labeled issues and swapping the underlying model mid-production do not require retraining cycles.

Overall, our findings suggest that retrieval-augmented few-shot prompting can achieve IRC performance competitive with LoRA fine-tuning. This makes it a practical alternative worth considering, particularly when hardware resources are constrained, labeled data is

XX:4 Retrieval-Augmented Few-Shot vs. Fine-Tuning for IRC

140 limited, or the deployment requires frequent model or data updates. The main contributions
141 are the following:

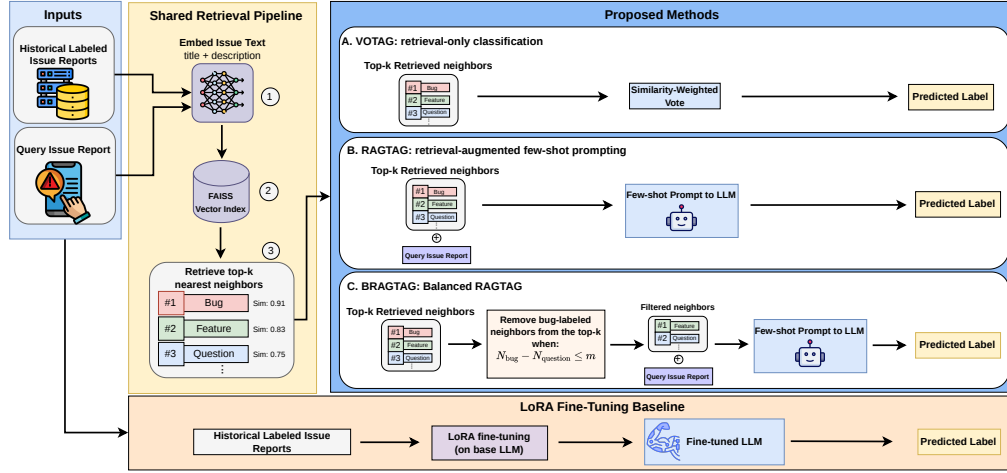
- 142 ■ Establishing a retrieval-only, non-parametric baseline for IRC that also serves as a fallback
143 mechanism for invalid LLM predictions (VOTAG).
- 144 ■ Conducting the first systematic evaluation of retrieval-augmented few-shot prompting for
145 IRC across different LLM scales and numbers of in-context examples (RAGTAG).
- 146 ■ Proposing a balanced retrieval technique for few-shot prompting that reduces question-to-
147 bug misclassifications without degrading performance on the other labels (BRAGTAG).
- 148 ■ Empirically comparing these proposed methods against the established LoRA fine-tuning
149 baseline on both classification performance and computational cost, in two data scope
150 settings (PS and PA). We find that retrieval-augmented few-shot prompting is competitive
151 with fine-tuning while using $11\times$ less labeled data per project and 33% less peak GPU
152 memory.

153 The remainder of this paper is organized as follows: Section 2 reviews prior work.
154 Section 3 explains our proposed methods and the fine-tuning baseline. Section 4 details the
155 experimental setup. Section 5 explains our evaluations and reports the results. Section 6
156 discusses failure modes, practical implications, and future work. Section 7 addresses the
157 potential threats to validity. Finally, Section 8 concludes the paper.

158 2 Related Work

159 **Automated Issue Report Classification (IRC).** Early IRC studies employed data mining
160 and ML classifiers [4, 30, 19]. However, these methods yielded limited performance due to their
161 shallow semantic understanding of issue report text [23]. To overcome this limitation, more
162 recent work have adopted transformer models for their stronger contextual understanding [11,
163 8, 44, 28, 29]. These studies typically fine-tune the models on historical issues in a supervised
164 setting. For instance, Izadi et al. [28] and Trautsch et al. [44] fine-tuned RoBERTa [38] and
165 seBERT [46] models, respectively, to classify issues into bugs, features, or questions. Although
166 transformer-based approaches achieve strong results, they rely on large amounts of training
167 data, which limits their generalizability to settings where labeled data is scarce [23, 28, 29, 11].
168 These challenges have motivated the exploration of LLMs for IRC. Their extensive pretraining
169 allows these models to be applied with little or no task-specific training, reducing the
170 dependence on the large labeled datasets that supervised classifiers require [6, 5, 14, 12, 13].
171 However, state-of-the-art LLM-based methods do not leverage this property. Instead, they
172 fine-tune the LLM with Low-Rank Adaptation (LoRA [25]) on labeled issue datasets to
173 achieve their best reported performance [23, 6, 5]. Although fine-tuning is effective, it
174 reintroduces the dependency on labeled training data, remains computationally expensive,
175 and must be repeated whenever the underlying model is swapped or as newly labeled issues
176 accumulate in the project [19, 30].

177 **Retrieval-Augmented Few-shot Prompting.** Studies show that few-shot prompting can
178 improve LLM performance in specialized tasks due to in-context learning [7, 34, 39, 27, 9].
179 Few-shot prompting has also been applied in IRC. Colavito et al. [12, 14] randomly selected
180 one or two labeled examples from their dataset and presented them in a few-shot prompt.
181 Few-shot performance, however, is sensitive to the quality of in-context examples. Research
182 shows that using retrieval techniques to select the most semantically relevant in-context
183 examples can outperform random selection [36, 51]. Since retrieval-augmented few-shot
184 prompting can enhance performance without the overhead of training, it can be a promising



■ **Figure 1** Overview of the four IRC approaches evaluated in this paper: VOTAG, RAGTAG, BRAGTAG, and the LoRA fine-tuning baseline.

185 alternative to LoRA fine-tuning IRC baseline. To the best of our knowledge, however, this
 186 method has not been systematically and empirically evaluated for IRC.

187 3 Approach

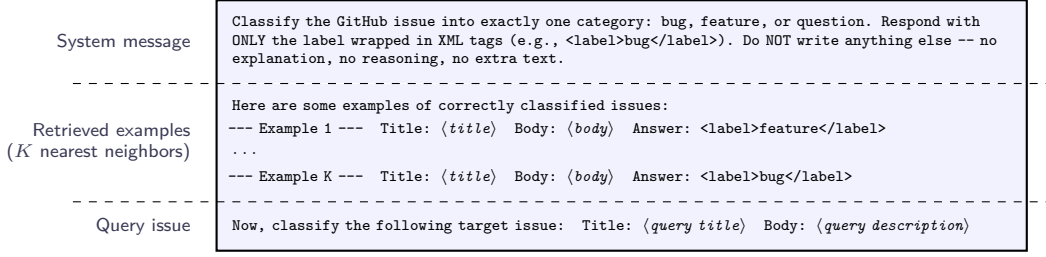
188 In this section, we describe the intuition behind, and the full implementation details of
 189 our three proposed retrieval-based IRC methods (VOTAG, RAGTAG, and BRAGTAG) as
 190 well as the faithful reproduction steps of the established LoRA fine-tuning baseline [23, 6].
 191 Figure 1 illustrates all four approaches to support the explanations. The remainder of the
 192 section is organized as follows: Section 3.1 explains the overall IRC objective. Section 3.2
 193 details the indexing and retrieval pipeline shared by our three proposed retrieval-based
 194 methods. Section 3.3 presents VOTAG, our non-parametric method of classifying the query
 195 directly from its nearest neighbors. Section 3.4 describes RAGTAG, our few-shot prompting
 196 technique that presents the nearest neighbors as classified examples in the prompt. Section 3.5
 197 elaborates on BRAGTAG’s balanced retrieval intervention on RAGTAG. Finally, Section 3.6
 198 details the faithful reproduction of the fine-tuning baseline.

199 3.1 Problem Formulation

200 Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or
 201 *question*. Only the issue’s title and description text are used for classification with no other
 202 signals from the issue tracker. The defined label space is adopted directly from prior IRC
 203 studies [5, 6, 28, 30, 4].

204 3.2 Indexing and Retrieval

205 To prepare the issues for retrieval based on textual similarity, we first embed their textual
 206 content (i.e., *title + description*). We do not embed the issue labels. Instead, we retain
 207 them as metadata associated with each embedded issue. To generate the embeddings, we use
 208 the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [41]. We selected this
 209 model due to its lightweight architecture, fast performance, and effectiveness on issue report



■ **Figure 2** The retrieval-augmented few-shot prompt structure used for RAGTAG and BRAGTAG.

text [26]. The embeddings are then indexed using the Faiss vector database library [17], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, Faiss calculates the cosine similarity of indexed issue reports to the query, and returns the top-K nearest neighbors, ordered by descending similarity.

3.3 VOTAG

VOTAG is our proposed non-parametric retrieval IRC method. VOTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors of the query issue report, then predicts the final label with a similarity-weighted vote on their labels. This approach follows the standard K-nearest-neighbor classification setup [15], with neighbors weighted by their similarity to the query [18].¹ Formally, for a query issue q with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^K$, where n_i is the i -th nearest neighbor with label l_i and cosine similarity s_i to q , the predicted label $\hat{y}$ is given by:

$$\hat{y} = \arg \max_{c \in \mathcal{L}} \sum_{i=1}^K s_i \cdot \mathbf{1}[l_i = c], \quad (1)$$

where $\mathcal{L}$ is the label set, and $\mathbf{1}[l_i = c]$ is the indicator function that equals 1 when neighbor i has label c and 0 otherwise. For each label VOTAG sums the cosine similarities of the neighbors that carry that label, and the arg max operator returns the label with the largest total sum. VOTAG breaks ties by returning the label of the most similar neighbor in the tied classes.

3.4 RAGTAG

RAGTAG is our proposed retrieval-augmented few-shot prompting approach for IRC. RAGTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors to the query issue, and present them as classified examples to the LLM in a few-shot prompt. This enables a more accurate classification, based on in-context learning [9]. The choice to present the top-K nearest neighbors as examples is based on two insights from prior research: First, retrieval-based example selection outperforms random selection [36]. Second, the most similar neighbors tend to share the same labels [15]; therefore, they are informative for the classification. We create the few-shot prompt in three parts as shown in Figure 2. First,

¹ Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

the system message frames the classification task and the label space. Second, the top- K nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title + description + label*. Finally, the query issue is appended to the end of the prompt, ensuring it receives high attention from the LLM [37]. $K = 0$ produces a zero-shot prompting baseline, similar to prior work [12].

We evaluate RAGTAG at top- K values in $\{0, 1, 3, 6, 9, 12, 15\}$. Details for the K value grid selection are provided in Sections 5.1 and 5.2. The maximum prompt sequence length is set to 8,192 tokens.² Roughly 30% of the context quota is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota, they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving all titles and labels (if any). The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format. This technique enables easy detection of the predicted label from the LLM’s generation, and reduces variance in LLM outputs caused by prompt format sensitivity [43]. We use a regular expression to extract the predicted label. Unparseable outputs are marked as *invalid*. Finally, models are loaded with Unsloth³ for memory efficiency, and the Temperature is set at 0.1 for deterministic classifications.

3.5 Balanced RAGTAG (BRAGTAG)

Our empirical evaluations in (Section 5.2) reveal that RAGTAG has the worst performance on the question label, and it frequently misclassifies true-question queries as bugs. Furthermore, LLM zero-shot results also show that the models are already biased towards predicting bug for true-question queries even before the examples are presented. Based on these insights, we hypothesize that presenting bug-labeled examples for true-question queries likely reinforces the question-to-bug misclassification trend. To address this, we propose a more balanced retrieval intervention, called balanced RAGTAG (BRAGTAG), that removes the bug-labeled neighbors whenever the query is a suspected question based on its nearest neighbors’ label distribution. (Section 5.2) shows that true-question queries retrieve almost balanced bug and question neighbors. Due to this balanced distribution, BRAGTAG removes the bug-labeled examples from the prompt whenever $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the top- k and m is a small positive margin. The margin is empirically calibrated on a holdout validation set. The empirical motivations for BRAGTAG and its margin calibration are fully detailed in (Section 5.2). Other than the balanced retrieval intervention, BRAGTAG inherits RAGTAG’s full setup and configurations.

3.6 LoRA Fine-Tuning Baseline

Recent IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [23, 6, 5], using Low-Rank Adaptation (LoRA) [25]. In these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt which includes the issue’s title and description as input, and its label as output. We adopt the instruction-style prompt template used in these prior LoRA fine-tuning studies [23, 6] verbatim, rather than designing our own, so that

² Maximum prompt sequence length is selected empirically. 8,192 tokens offered the best trade-off between classification accuracy and inference speed across our preliminary experiments.

³ <https://unsloth.ai>

the baseline faithfully reproduces their established methodology. At inference, the trained model receives the test issue in the same format, and predicts the empty label field. Regular expressions are used to extract the prediction from the output, and unparseable outputs are marked as *invalid*. We use this established methodology as our fine-tuning baseline.

Training hyperparameters follow prior work [23, 6]: The LoRA rank and alpha are set to 16, with a learning rate of 2×10^{-4} and the paged AdamW 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient accumulation steps of 16. The maximum prompt sequence length is set to 2,048 tokens, since 94.9% of the test issues in the dataset will not be truncated at this length. Issues that exceed this length get truncated from the right side of their description text to fit. Similar to RAGTAG and BRAGTAG, the models are loaded with Unsloth and the Temperature is set at 0.1.

4 Experimental Setup

Dataset and Settings. We use the dataset introduced in Heo et al. [23] for our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible`, `bitcoin`, `dart-lang`, `roslyn`, `react`, `flutter`, `kubernetes`, `TypeScript`, `vscode`, `opencv`, and `tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug*, *feature*, and *question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project. We also adopt Heo et al.’s project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC performance. In the PS setting, each project’s train and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAGTAG and VOTAG only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline.

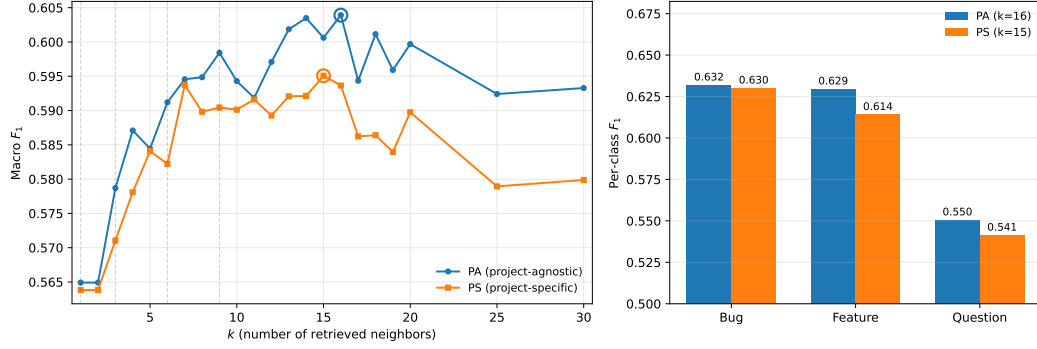
Models. We use the Qwen2.5-Instruct model family [50] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model with different parameter sizes enables analysis of the model scale effect in LLM-based IRC approaches.

Evaluation Metrics. We report the per-class F_1 and the macro-averaged F_1 over the three classes (*bug*, *feature*, *question*). The macro F_1 is calculated over the total 3,300-issue test set in all settings. We also report the rate of outputs that fail to parse to a valid label (*invalid rate*). Invalid outputs are counted as incorrect predictions in all F_1 computations. For statistical significance checks, we report paired bootstrap 95% confidence intervals on the macro F_1 differences between methods.

Hardware Setup. The experiments were conducted on a machine with one NVIDIA L40 GPU (48 GB VRAM), 8 allocated CPU cores from an Intel Xeon Scalable processor, and 64 GiB of system RAM.

5 Evaluations

In this section, we present the results of our experiments and answer the four research questions posed in Section 1, one per subsection.



■ **Figure 3 (left)** VOTAG macro F_1 as a function of k in both PS and PA settings. Circles mark peak performance. **(right)** Per-class F_1 at each setting's best k ($k=15$ for PS, $k=16$ for PA).

5.1 RQ1: Effectiveness of Retrieval-Only IRC

We evaluate VOTAG's performance on both PA and PS settings at k values 1 to 30. The resulting macro F_1 scores are shown in Figure 3. VOTAG's best macro F_1 is 0.595 ($k=15$) in PS and 0.604 ($k=16$) in PA. When adding neighbors beyond the peak, PS loses 0.015 and PA loses 0.011 by $k=30$. PA consistently outperforms PS at every k . However, the performance gap is marginal: averaging 0.007 macro F_1 , with a maximum of 0.015 at $k=18$. This similar performance is due to the large overlap of the top- k neighbors in both settings. Specifically, PA and PS share 84–90% of the top- k neighbors across $k \in [3, 30]$. Therefore, even when using the full 3,300-issue retrieval index, most of the retrieved neighbors are from the same project. The question class consistently has the weakest performance among the three labels at every k in both settings. PS per-class F_1 at best k is 0.630 for bug, 0.614 for feature, and 0.541 for question classes. The corresponding PA values are 0.632, 0.629, and 0.550 (Figure 3 right panel). Additionally, questions are more frequently misclassified as bugs: 32% of question issues are predicted as bugs in both settings, while 18% (PS) and 17% (PA) are predicted as features. Since VOTAG predicts the label with the most accumulated similarity weight, this misclassification asymmetry indicates that question issues are more similar to bugs than to features in the embedding space.

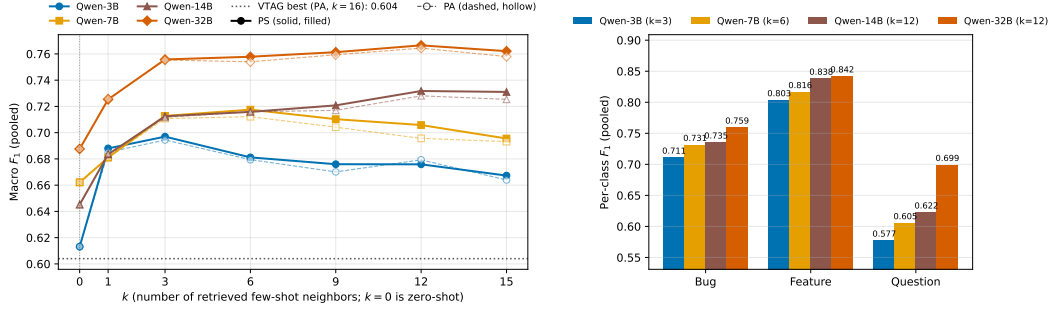
Retrieval alone establishes a non-parametric baseline with a peak macro F_1 of 0.604 and bounds the useful number of in-context examples at $k \approx 15$.

5.2 RQ2: Effectiveness of Retrieval-Augmented Few-Shot IRC

We evaluate RAGTAG at k values $\{0, 1, 3, 6, 9, 12, 15\}$ on both PA and PS settings, where $k=0$ is the zero-shot baseline [12]. k value selection is informed by VOTAG's peak performance around $k=15$ in the previous section. Figure 4 (left panel) shows the macro F_1 of every model across this grid for both settings.

PS marginally outperforms PA at almost every (model, $k \geq 1$) pair evaluated (zero-shot is identical between settings since no retrieval is performed). The close performance gap is due to the same top- k neighbor overlap observed in the VOTAG analysis. The marginal improvement is significant because PS uses $11 \times$ less retrieval data per project: each PS index covers only its own 300 retrieval issues, while PA uses the full 3,300 across all 11 projects. Given this practical advantage, we conduct the rest of our RAGTAG analysis on the PS

XX:10 Retrieval-Augmented Few-Shot vs. Fine-Tuning for IRC



■ **Figure 4 (left)** RAGTAG macro F_1 as a function of k for all four Qwen sizes. $k=0$ is the zero-shot baseline. The dotted horizontal line marks VOTAG’s best (0.604, PA $k=16$). **(right)** Per-class F_1 for each model size at its best RAGTAG-PS configuration.

349 setting and report PS numbers throughout this section.

350 The best k grows with model size: Qwen-3B peaks at $k=3$ (macro $F_1 = 0.697$), Qwen-7B
 351 at $k=6$ (0.718), and both Qwen-14B and Qwen-32B at $k=12$ (0.732 and 0.767 respectively).
 352 This shows that larger models can use the context from additional examples more effectively.
 353 However, adding neighbors past $k=12$ has no benefit and slightly degrades performance:
 354 every model loses 0.001–0.015 macro F_1 going from $k=12$ to $k=15$.

355 Every RAGTAG configuration with $k \geq 1$ outperforms the zero-shot baseline. At each
 356 model’s peak, the gap over zero-shot averages +0.076 macro F_1 and ranges from +0.055
 357 (Qwen-7B) to +0.087 (Qwen-14B). This shows that LLMs benefit substantially from the
 358 labeled examples for IRC. Every RAGTAG configuration also outperforms VOTAG’s best
 359 result (0.604 at PA $k=16$), with the smallest margin from Qwen-3B zero-shot (+0.009 F_1)
 360 and the largest from Qwen-32B PS at $k=12$ (+0.163 F_1). These results show that adding
 361 the LLM reasoning improves IRC on top of pure retrieval.

362 Similar to VOTAG, the question class consistently has the weakest performance among
 363 the three labels across all RAGTAG settings. Figure 4 (right panel) shows per-class F_1 at
 364 each model’s best k . Even the smallest gaps between question and the other labels, observed
 365 at Qwen-32B with $k=12$, are noticeable: question’s macro F_1 is 0.060 below bug’s and 0.143
 366 below feature’s. Similar to the VOTAG observations, questions tend to be more frequently
 367 misclassified as bugs. At zero-shot, across all four model sizes, 46–59% of true-question issues
 368 are predicted as bugs while only 5–16% are predicted as features. RAGTAG narrows this
 369 misclassification asymmetry but does not eliminate it: at each model’s best k , 26–36% of
 370 question issues are still predicted as bugs and only 9–15% as features.

371 These results show that the LLMs are already prone to misclassifying question issues as
 372 bugs, even before the top- k examples are used. Therefore, presenting bug-labeled examples for
 373 true-question query issues likely reinforces these misclassifications. Based on this intuition, we
 374 propose a more balanced retrieval technique as an intervention that removes the bug-labeled
 375 neighbors whenever the query is a suspected question. We base the suspicion signal on the
 376 nearest neighbors’ label distribution and calibrate it without including the test set. From
 377 each project’s total 300 training issues, we assign 30 issues stratified by label as queries and
 378 270 as the retrieval index, mirroring the PS setting. On this validation split, true-question
 379 queries retrieve almost balanced bug and question examples: averaged across $k \in [1, 15]$, the
 380 mean of $N_{\text{bug}} - N_{\text{question}}$ is -1.36 . We therefore treat a query as a suspected question when
 381 $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the top- k

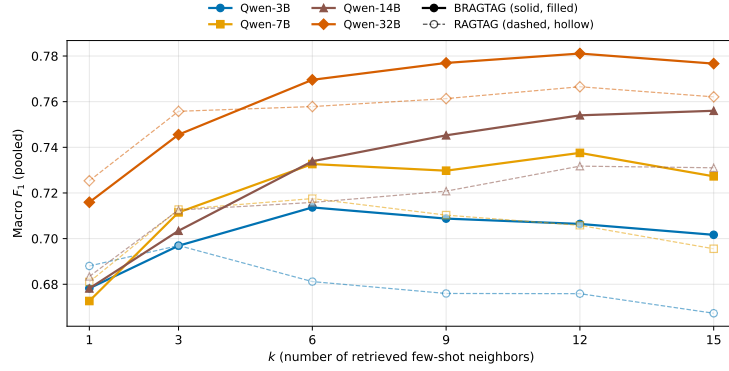


Figure 5 Macro F_1 as a function of k across all model sizes for both RAGTAG and BRAGTAG (project-specific setting)

and m is a small positive margin. Sweeping $m \in \{1, 2, 3, 4, 5\}^4$ on the validation split and averaging across $k \in [1, 15]$, $m=3$ had a more favorable trade-off against the other values, firing for 89.1% of true-question and 56.7% of true-bug queries. We adopt $m=3$ and refer to the resulting method as **Balanced RAGTAG** (BRAGTAG).

Every Retrieval-augmented few-shot prompting (RAGTAG) configuration with $k \geq 1$ consistently outperforms the zero-shot and VOTAG baselines. The best k grows with model size but plateaus within the $k \approx 15$ bound established by VOTAG. RAGTAG is weakest on the question class and frequently misclassifies questions as bugs, motivating a more balanced retrieval method.

5.3 RQ3: Effectiveness of the Balanced Retrieval method

Since the PS setting produced the best results for RAGTAG, we use PS as the default for evaluating BRAGTAG on the same $k \in \{1, 3, 6, 9, 12, 15\}$ grid. Figure 5 shows macro F_1 vs k for both methods across all four model sizes. BRAGTAG outperforms RAGTAG at every $k \geq 6$. At smaller $k \in \{1, 3\}$, BRAGTAG slightly underperforms by 0.001–0.010 macro F_1 , because the neighbor balancing intervention removes every retrieved example too often, practically reverting the query to zero-shot (40% of queries at $k=1$ and 18% at $k=3$).

BRAGTAG’s best k almost always increases compared to RAGTAG: Qwen-3B from $k=3$ to $k=6$, Qwen-7B from $k=6$ to $k=12$, Qwen-14B from $k=12$ to $k=15$, and Qwen-32B unchanged at $k=12$. When the trigger fires, BRAGTAG removes all bug examples from the top- k . Therefore, A higher initial k is needed to leave enough remaining context for the LLM to use. At each method’s best k , BRAGTAG outperforms RAGTAG on every model. The paired bootstrap 95% CI on the (BRAGTAG – RAGTAG) macro F_1 difference excludes zero in all four cells:⁵ Qwen-3B +0.017 [+0.003, +0.031], Qwen-7B +0.020 [+0.007, +0.032], Qwen-14B +0.024 [+0.014, +0.034], and Qwen-32B +0.014 [+0.007, +0.022]. Table 1 reports the full best configuration comparison.

The performance gain comes from improvements in the question and bug classes, with minimal impact on feature. Table 1 reports per-class F_1 for each method at its best k .

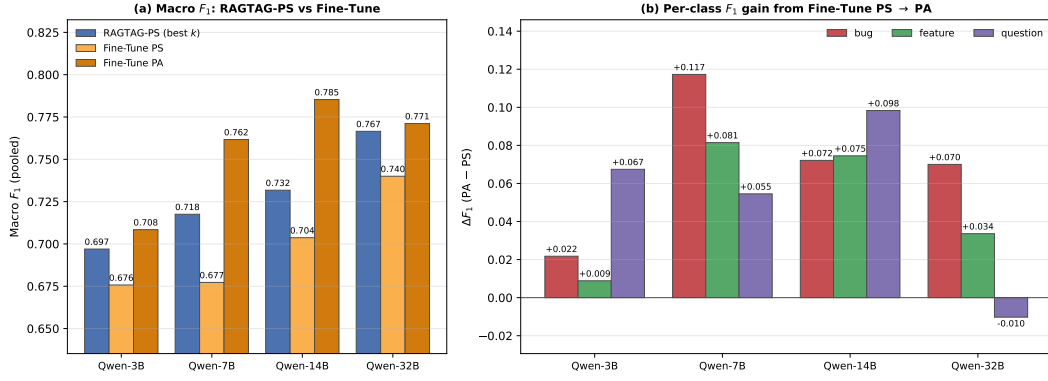
⁴ We cap the sweep at $m=5$, since it is the 95th percentile of $N_{\text{bug}} - N_{\text{question}}$ on true-question queries.

⁵ 1,000 paired bootstrap resamples per model.

XX:12 Retrieval-Augmented Few-Shot vs. Fine-Tuning for IRC

■ **Table 1** RAGTAG vs BRAGTAG at each model’s best k (PS, pooled).

Model	Method	k^*	Macro F_1	F_1^{bug}	F_1^{feat}	F_1^{q}	Invalid rate
Qwen-3B	RAGTAG	3	0.697	0.711	0.803	0.577	0.2%
	BRAGTAG	6	0.714	0.693	0.803	0.645	1.8%
Qwen-7B	RAGTAG	6	0.718	0.731	0.816	0.605	3.5%
	BRAGTAG	12	0.738	0.742	0.805	0.666	3.8%
Qwen-14B	RAGTAG	12	0.732	0.735	0.838	0.622	4.5%
	BRAGTAG	15	0.756	0.745	0.840	0.683	4.7%
Qwen-32B	RAGTAG	12	0.767	0.759	0.842	0.699	4.6%
	BRAGTAG	12	0.781	0.775	0.840	0.729	4.0%



■ **Figure 6** (a) Macro F_1 of RAGTAG with its best k under the PS data scope, compared with the LoRA fine-tuning baseline under both PS and PA data scopes. (b) Per-class F_1 change from PS to PA LoRA fine-tuning ($\Delta F_1 = \text{PA} - \text{PS}$) for each class and model size.

Question-class F_1 improves by 0.030–0.068 across all four model sizes. Bug-class F_1 remains within 0.018 of RAGTAG, with a slight decrease for Qwen-3B and slight gains for the larger models. Thus, removing bug neighbors in selected cases does not reduce overall bug performance. Feature-class F_1 is essentially unchanged. The question-to-bug misclassification rate also decreases: at each model’s best k , RAGTAG misclassifies 26–36% of true-question issues as bugs, whereas BRAGTAG reduces this rate to 19–27%. Averaged across $k \in [1, 15]$, the rate drops from 31–40% for RAGTAG to 24–36% for BRAGTAG. Question-to-feature misclassification remains unaffected. These results confirm that BRAGTAG’s improvement comes from a targeted reduction in question-to-bug misclassification.

Outputs from both approaches that failed to parse to a valid label are marked as *invalid* and counted as incorrect. Averaged across $k \in [1, 15]$, BRAGTAG has a lower invalid rate than RAGTAG (2.3% vs 3.0% across the four models).

The balanced retrieval method (BRAGTAG), outperforms RAGTAG at every $k \geq 6$. The gain comes from a targeted reduction of question-to-bug misclassifications, raising question F_1 by 0.030–0.068 without sacrificing bug or feature performance.

418 5.4 RQ4: Few-Shot Prompting vs. LoRA Fine-Tuning for IRC

■ **Table 2** RAGTAG, BRAGTAG, and Fine-Tune at each method’s best configuration (RAGTAG/BRAGTAG at their best k on PS, Fine-Tune on PA). Macro F_1 and per-class F_1 are computed on raw predictions. The +VOTAG column reports macro F_1 with VOTAG as fallback for the invalid LLM outputs. Train and Inference are run-times on a single GPU with Total as their sum. RAM is the peak GPU memory observed

Model	Method	k^*	Macro F_1	+VOTAG	F_1^{bug}	F_1^{feat}	F_1^{q}	RAM (GB)	Train (h)	Infer (h)	Total (h)
Qwen-3B	RAGTAG	3	0.697	0.696	0.711	0.803	0.577	2.9	–	0.18	0.18
	BRAGTAG	6	0.714	0.720	0.693	0.803	0.645	2.9	–	0.22	0.22
	Fine-Tune	–	0.708	0.711	0.736	0.790	0.599	5.5	0.31	0.11	0.42
Qwen-7B	RAGTAG	6	0.718	0.729	0.731	0.816	0.605	6.8	–	0.50	0.50
	BRAGTAG	12	0.738	0.751	0.742	0.805	0.666	6.8	–	0.60	0.60
	Fine-Tune	–	0.762	0.762	0.762	0.846	0.678	9.9	0.48	0.10	0.58
Qwen-14B	RAGTAG	12	0.732	0.746	0.735	0.838	0.622	12.6	–	1.37	1.37
	BRAGTAG	15	0.756	0.771	0.745	0.840	0.683	12.6	–	1.35	1.35
	Fine-Tune	–	0.785	0.785	0.792	0.828	0.736	16.7	1.49	0.22	1.71
Qwen-32B	RAGTAG	12	0.767	0.779	0.759	0.842	0.699	22.3	–	4.62	4.62
	BRAGTAG	12	0.781	0.792	0.775	0.840	0.729	22.3	–	4.15	4.15
	Fine-Tune	–	0.771	0.771	0.791	0.853	0.669	31.5	2.28	0.44	2.71

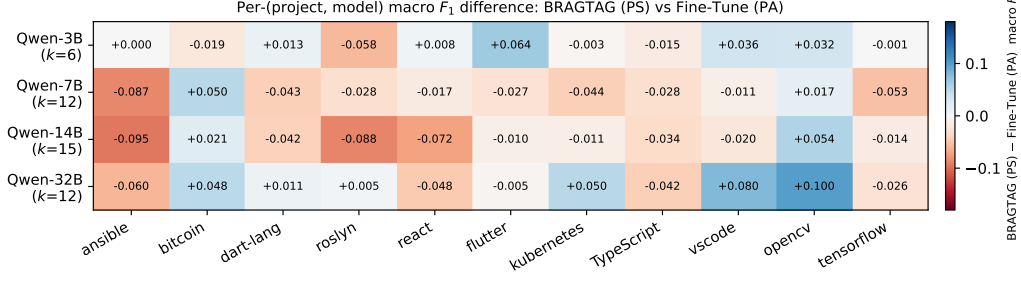
Classification Performance. We fine-tune all models under both PA and PS data scope settings. Figure 6(a) compares macro F_1 for LoRA fine-tuning in both settings against RAGTAG in the PS setting at each model’s best k . Project-agnostic LoRA fine-tuning consistently outperforms project-specific LoRA fine-tuning across all model sizes, with gains of +0.033 for Qwen-3B, +0.084 for Qwen-7B, +0.082 for Qwen-14B, and +0.031 macro F_1 for Qwen-32B. This trend is consistent with prior findings on LLM fine-tuning for IRC [23].

RAGTAG under the PS data scope outperforms project-specific LoRA fine-tuning at every model size, with macro F_1 gains ranging from +0.021 for Qwen-3B to +0.040 for Qwen-7B. These results show that, using only 300 labeled issue reports per project, retrieval-augmented few-shot prompting is more effective than LoRA fine-tuning for IRC. Figure 6(b) shows where the additional PA training data benefits LoRA fine-tuning most. Averaged across the four model sizes, bug-class F_1 increases by 0.070 when moving from PS to PA fine-tuning, while question-class and feature-class F_1 increase by 0.053 and 0.050, respectively. Qwen-32B is the only exception: its question-class F_1 decreases by 0.010, while its bug-class F_1 increases by 0.070.

Table 2 reports macro and per-class F_1 for each method at its best configuration. Overall, RAGTAG achieves competitive macro F_1 relative to LoRA fine-tuning. Aggregated across the four model sizes (13,200 paired predictions), LoRA fine-tuning exceeds RAGTAG by 0.029 macro F_1 (paired bootstrap 95% CI $[-0.037, -0.022]$). The gap varies by model size: RAGTAG is closest to LoRA fine-tuning for the smallest and largest models and farther behind for the intermediate sizes. The per-model macro F_1 differences (RAGTAG – LoRA fine-tuning), with paired bootstrap 95% CIs in brackets, are -0.011 $[-0.028, +0.007]$ for Qwen-3B, -0.044 $[-0.059, -0.029]$ for Qwen-7B, -0.053 $[-0.068, -0.039]$ for Qwen-14B, and -0.004 $[-0.020, +0.009]$ for Qwen-32B. The confidence intervals for Qwen-3B and Qwen-32B include zero, indicating that RAGTAG and LoRA fine-tuning are not statistically distinguishable at these model sizes.

BRAGTAG’s balanced retrieval strategy narrows this gap: the aggregate macro F_1 difference between BRAGTAG and LoRA fine-tuning is -0.010 (BRAGTAG – fine-tuning). However, LoRA fine-tuning does not consistently outperform BRAGTAG across projects. Figure 7 reports the per-(project, model) macro F_1 difference between BRAGTAG under the PS data scope and PA LoRA fine-tuning. BRAGTAG outperforms LoRA fine-tuning in 16 of the 44 project–model cells, including all four model sizes on `opencv` and three of four on `bitcoin`. As with RAGTAG, BRAGTAG performs better relative to LoRA fine-tuning at

XX:14 Retrieval-Augmented Few-Shot vs. Fine-Tuning for IRC



■ **Figure 7** Per-(project, model) macro F_1 difference between BRAGTAG under the PS data scope and PA LoRA fine-tuning across the 11 projects and four Qwen2.5-Instruct model sizes. Each BRAGTAG result uses the corresponding model’s best k .

the smallest and largest model sizes: it is statistically tied with LoRA fine-tuning at Qwen-3B and Qwen-32B, while LoRA fine-tuning leads at the mid-sized models. The per-model macro F_1 differences (BRAGTAG – LoRA fine-tuning), with paired bootstrap 95% CIs in brackets, are +0.006 [−0.012, +0.024] for Qwen-3B, −0.024 [−0.039, −0.009] for Qwen-7B, −0.029 [−0.044, −0.013] for Qwen-14B, and +0.010 [−0.005, +0.024] for Qwen-32B.

BRAGTAG achieves the highest question-class F_1 for Qwen-3B and Qwen-32B, whereas LoRA fine-tuning leads for Qwen-7B and Qwen-14B. LoRA fine-tuning achieves the highest bug-class F_1 across all four model sizes. For the feature class, BRAGTAG leads for Qwen-3B and Qwen-14B, while LoRA fine-tuning leads for Qwen-7B and Qwen-32B. These results indicate that BRAGTAG remains competitive with LoRA fine-tuning through its targeted question-to-bug correction, which improves question F_1 without degrading bug or feature performance. Averaged across the four model sizes, BRAGTAG is also the most class-balanced approach, yielding the lowest per-class F_1 standard deviation (0.058, compared with 0.067 for LoRA fine-tuning and 0.082 for RAGTAG) and the highest worst-class F_1 floor (0.681, compared with 0.671 and 0.626, respectively).

LoRA fine-tuning produces fewer invalid outputs than RAGTAG and BRAGTAG under both data scopes, because the model learns the required output format during training. Averaged across the four model sizes, LoRA fine-tuning has invalid-output rates of 0.28% in the PA setting and 0.38% in the PS setting, compared with 3.0% for RAGTAG and 2.3% for BRAGTAG averaged across $k \in [1, 15]$. In an ideal triage scenario, every issue report should receive a valid label; however, a known limitation of LLMs is that they can produce outputs that do not comply with task instructions [43, 3]. To address invalid predictions, we use VOTAG as a fallback mechanism. VOTAG never produces an invalid label and relies on the neighbors already retrieved for RAGTAG/BRAGTAG, adding no LLM inference cost. For a fair comparison, we apply the fallback to all three methods under their best data scopes: VOTAG-PS for RAGTAG/BRAGTAG and VOTAG-PA for LoRA fine-tuning. Under this protocol, BRAGTAG reaches statistical equivalence with LoRA fine-tuning, passing TOST at $\delta=0.01$ (bootstrap 95% CI [−0.007, +0.008]⁶).

Computation and Data Cost. The methods impose different hardware requirements. Table 2 reports peak GPU memory by model size. RAGTAG and BRAGTAG have identical GPU memory requirements, which increase with model size, whereas LoRA fine-tuning

⁶ 1,000 paired bootstrap resamples per model for both RAGTAG vs. LoRA fine-tuning and BRAGTAG vs. LoRA fine-tuning comparisons.

requires more memory at every size due to training overheads, including gradients, optimizer state, and activations retained for the backward pass. Averaged across the four model sizes, RAGTAG and BRAGTAG require 33% less peak GPU memory than LoRA fine-tuning, with reductions of 47%, 31%, 25%, and 29% for Qwen-3B, Qwen-7B, Qwen-14B, and Qwen-32B, respectively.

Notably, RAGTAG and BRAGTAG in the PS setting retrieve from only 300 labeled issue reports per project, whereas PA LoRA fine-tuning uses the full set of 3,300 labeled issue reports across all 11 projects to achieve its best configuration. Thus, despite using $11\times$ less labeled data per project, RAGTAG remains competitive with LoRA fine-tuning, and BRAGTAG closes the gap to TOST equivalence at $\delta=0.02$. With the VOTAG fallback for invalid LLM outputs, this further tightens to statistical equivalence at $\delta=0.01$.

RAGTAG outperforms LoRA fine-tuning by 0.021–0.040 macro F_1 at every model size with project-specific data scope. Fine-tuning outperforms RAGTAG only when trained on $11\times$ more labeled data via cross-project pooling (PA), and leads by 0.029 aggregate macro F_1 . BRAGTAG narrows the aggregate gap to -0.010 , reaching statistical equivalence with fine-tuning once a VOTAG fallback handles invalid LLM outputs. Finally, retrieval-augmented few-shot requires 33% less peak GPU memory than fine-tuning.

6 Discussion

6.1 Failure Analysis

In this section we conduct a manual qualitative analysis to better understand the misclassification patterns across the explored IRC methods. We inspect the misclassifications produced by Qwen-32B (the strongest model) under each method’s best configuration. From the full set of misclassifications produced by RAGTAG, BRAGTAG, and LoRA fine-tuning, we randomly sampled 50 misclassified instances per method, yielding 150 instances in total. Each method’s sample includes 30 true-question, 10 true-bug, and 10 true-feature errors. The 30/10/10 split mimics the empirical class distribution of misclassifications pooled across the three methods. Each case was analyzed by two authors independently, with any discrepancies resolved through discussions. The observed failure categories and their frequencies are as follows:

Wrong template format (77.33%). Issue reports whose content and ground-truth label contradicts their chosen template and structure. All of these errors are from questions and feature requests that the user filed under a bug-report template (*Steps to reproduce*, *Expected*, *Actual*, *System Information*). Consequently, the structure of the bug template misleads the LLM classifier towards predicting bug, regardless of the issue’s content. For example, `vscode#193985`⁷ is an actual support question that opens with a literal `Type:Bug` because it uses VS Code’s bug template.

Hybrid intent (13.33%). Issues that could have multiple defensible labels due to their combined intents. These errors mostly take place when a fix or a feature request is layered in a question or bug. For example, in `kubernetes#120364`⁸ the author both reports a concrete

⁷ <https://github.com/microsoft/vscode/issues/193985>

⁸ <https://github.com/kubernetes/kubernetes/issues/120364>

bug and in the same body explicitly requests a new feature: “*Please, give us multiarchitecture image! and fix monoarchitecture images too!*”

Label noise (9.33%). Issues whose ground-truth label does not match its content. For example, `tensorflow#61710`⁹ is titled “*Will there a MLP model in the future version?*” which requests a new API feature for building MLPs. Yet the dataset labels it as a bug.

We also sampled 30 *invalid* predictions each from RAGTAG and BRAGTAG (60 total) and observed three failure modes. In roughly 73% of cases, the model either continued to generate issue-body content or its own reasoning. Another 23% were hallucinations such as repeated digits and punctuations. The remaining 3% are parsable labels that fall outside the three valid classes (e.g., `documentation`).

6.2 Key Insights and Practical Implications

The practicality of context retrieval for IRC. Our results indicate that RAG-based approaches offer a more practical solution for IRC than fine-tuning. With $11\times$ less labeled data per project and 25–47% less peak GPU memory, RAGTAG performs competitively against fine-tuning (trails by 0.029 in aggregate macro F1), and BRAGTAG closes the gap to TOST equivalence at $\delta=0.02$, tightening to statistical equivalence at $\delta=0.01$ under the VOTAG fallback mechanism for invalid LLM outputs (Section 5.4). RAG-based methods offer deployment advantages over fine-tuning as well: adding a new labeled issue report becomes an incremental index update rather than a retraining cycle, and the model can be swapped in production with no project-specific retraining. For extremely resource-constrained scenarios, even VOTAG PS is a defensible baseline: it reaches macro $F_1 = 0.595$ ($k=15$), within 0.018 of Qwen-3B’s zero-shot 0.613 (Section 5.1, Section 5.2), at a fraction of the GPU cost (≈ 240 MB vs 2.9 GB).

Combining classification and validation. Our failure analysis (Section 6.1) shows that users frequently file support questions and feature requests under the bug-report template, which can bias the classifier towards predicting bug regardless of the content. This emphasizes the importance of refining the new LLM-assisted triage pipelines now in place on platforms such as GitHub [20] and GitLab [21]. In these pipelines, the reporter describes the report to an LLM, which then suggest the correct template and labels. Report validation is also important since mislabeling and duplication is a common problem among contributors[4, 40]. Studies have already explored validation for bug reports [31, 48, 2, 16]. However, combining classification and validation in a cohesive workflow is underexplored.

7 Threats to Validity

Internal Validity. BRAGTAG’s margin uses the same empirically derived fixed value across all models and projects with no per-project or per-model tuning, and is not framed as a universally optimal constant. Furthermore, the choice of hyperparameters and prompt format can also pose an internal threat to validity as different hyperparameters and prompts can change fine-tuning performance [23, 6].

External Validity. Issue report text can vary substantially in writing style, vocabulary, and structure across software projects [3], which may threaten the generalizability of our findings. Another threat to external validity is generalizability across settings. We note

⁹ <https://github.com/tensorflow/tensorflow/issues/61710>

that many configurations, including LLM architecture, prompt structure, embedding model, and retrieval technique, can each lead to different results. Assessing how far our findings generalize to other settings is itself an important question, but one that falls outside this paper’s scope and warrants its own dedicated studies.

Construct Validity. The inherent randomness of LLM outputs can pose a potential threat to construct validity. The reported performance metrics can vary between different runs due to LLM randomness. To mitigate this variance, we ran the experiments three times and report the average.

8 Conclusion and Future Work

In this study, we investigated retrieval-augmented few-shot prompting as a cost-efficient alternative to LoRA fine-tuning for issue report classification (IRC). Given a query issue, we retrieved its top- k nearest neighbors from historical issue reports and used them in three proposed approaches: RAGTAG, which presents the neighbors as labeled examples in a few-shot prompt; BRAGTAG, a RAGTAG variant that removes bug-labeled neighbors when the retrieved label distribution suggests that the query is likely a question; and VOTAG, which predicts the label through a weighted vote over the neighbors’ labels, serving as a non-parametric baseline.

We evaluated all three approaches against a LoRA fine-tuning baseline on an 11-project benchmark using Qwen2.5-Instruct models at four sizes. The evaluation considered both project-specific and project-agnostic data scope settings. RAGTAG in the project-specific setting outperforms project-specific LoRA fine-tuning by 0.021–0.040 macro F_1 across all four model sizes and trails project-agnostic LoRA fine-tuning by only 0.029 macro F_1 in aggregate, despite using $11\times$ less labeled data per project. BRAGTAG in the project-specific setting further closes this gap to TOST equivalence at $\delta=0.02$, tightening to statistical equivalence at $\delta=0.01$ under the VOTAG fallback protocol for invalid LLM outputs. Averaged across all models, retrieval-augmented few-shot prompting also requires 33% less peak GPU memory than LoRA fine-tuning. Taken together, these results validate our hypothesis: under the studied conditions, low-cost RAG-based approaches can achieve competitive performance with state-of-the-art fine-tuning-based IRC methods, confirming their potential as cost-efficient alternatives.

These findings position retrieval-augmented few-shot prompting as a practical IRC method, particularly when hardware resources are constrained, labeled data is limited, or deployment requires frequent model or data updates. As this work constitutes a case study on the Qwen2.5-Instruct family, future work can focus on the generalizability of the hypothesis by extending it to other LLM architectures to further strengthen or bound the conclusions drawn here. Additional directions include examining imbalanced label distributions, minority-class scenarios, multi-label classification, and other issue tracking platforms such as Jira and Bugzilla. Further research should also explore stronger retrieval-side interventions, such as LLM-based filtering or re-ranking of retrieved neighbors, as well as the effects of different embedding models and similarity metrics on retrieval quality. Finally, future studies should investigate alternative prompt structures, including advanced strategies such as Chain-of-Thought prompting [32].

600 **Data Availability.** Our replication package [1] contains the code, data, and all necessary
601 materials to reproduce the results presented in this paper.

602 — References —

- 603 1 Replication package: can retrieval-augmented few-shot prompting match lora fine-tuning
604 for issue report classification? an empirical study. [https://figshare.com/s/](https://figshare.com/s/2c7a6db860c611246774)
605 [2c7a6db860c611246774](https://figshare.com/s/2c7a6db860c611246774). Accessed: 05-18-2026.
- 606 2 Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and
607 Martin Hirzel. Otter: Generating tests from issues to validate swe patches. *arXiv preprint*
608 *arXiv:2502.05368*, 2025.
- 609 3 Arshia Akhavan, Alireza Hoseinpour, Abbas Heydarnoori, Hamid Bagheri, and Mehdi Keshani.
610 Linkanchor: An autonomous llm-based agent for issue-to-commit link recovery, 2026. URL:
611 <https://arxiv.org/abs/2508.12232>, arXiv:2508.12232.
- 612 4 Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël
613 Guéhéneuc. Is it a bug or an enhancement? a text-based approach to classify change requests.
614 In *Proceedings of the 2008 conference of the center for advanced studies on collaborative*
615 *research: meeting of minds*, pages 304–318, 2008.
- 616 5 Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A. Gerosa. Applying
617 large language models api to issue classification problem, 2024. URL: [https://arxiv.org/](https://arxiv.org/abs/2401.04637)
618 [abs/2401.04637](https://arxiv.org/abs/2401.04637), arXiv:2401.04637.
- 619 6 Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A Gerosa. Applying
620 large language models to issue classification: Revisiting with extended data and new models.
621 *Science of Computer Programming*, page 103333, 2025.
- 622 7 Maram Assi, Safwat Hassan, and Ying Zou. Llm-cure: Llm-based competitor user review
623 analysis for feature enhancement. *ACM Transactions on Software Engineering and Methodology*,
624 35(4):1–25, 2026.
- 625 8 Shikhar Bharadwaj and Tushar Kadam. Github issue classification using bert-style models. In
626 *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*,
627 pages 40–43, 2022.
- 628 9 Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners.
629 *Advances in neural information processing systems*, 33:1877–1901, 2020.
- 630 10 Gemma Catolino, Fabio Palomba, Andy Zaidman, and Filomena Ferrucci. Not all bugs are
631 the same: Understanding, characterizing, and classifying bug types. *Journal of Systems and*
632 *Software*, 152:165–181, 2019.
- 633 11 Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Issue report classification using
634 pre-trained language models. In *Proceedings of the 1st International Workshop on Natural*
635 *Language-based Software Engineering*, pages 29–32, 2022.
- 636 12 Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Benchmarking large language models
637 for automated labeling: The case of issue report classification. *Information and Software*
638 *Technology*, page 107758, 2025.
- 639 13 Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. Large language
640 models for issue report classification. In *Ital-IA*, pages 18–23, 2024.
- 641 14 Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. Leveraging gpt-like
642 llms to automate issue labeling. In *Proceedings of the 21st International Conference on Mining*
643 *Software Repositories*, pages 469–480, 2024.
- 644 15 Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on*
645 *information theory*, 13(1):21–27, 1967.
- 646 16 Emre Dinç and Eray Tüzün. Judge the votes: A system to classify bug reports and give
647 suggestions. In *2025 2nd IEEE/ACM International Conference on AI-powered Software*
648 *(AIware)*, pages 01–10. IEEE, 2025.

- 649 17 Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-
650 Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE*
651 *Transactions on Big Data*, 2025.
- 652 18 Sahibsingh A Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on*
653 *Systems, Man, and Cybernetics*, (4):325–327, 1976.
- 654 19 Qiang Fan, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. Where is the road for issue
655 reports classification based on text mining? In *2017 ACM/IEEE International Symposium on*
656 *Empirical Software Engineering and Measurement (ESEM)*, pages 121–130. IEEE, 2017.
- 657 20 GitHub. GitHub Copilot. <https://github.com/features/copilot>, 2025.
- 658 21 GitLab. GitLab Duo. <https://about.gitlab.com/gitlab-duo/>, 2025.
- 659 22 Luiz Gomes, Ricardo da Silva Torres, and Mario Lúcio Côrtes. Bert-and tf-idf-based feature
660 extraction for long-lived bug prediction in floss: A comparative study. *Information and*
661 *Software Technology*, 160:107217, 2023.
- 662 23 Jueun Heo and Seonah Lee. A study on applying large language models to issue classification.
663 In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, pages
664 1–11. IEEE Computer Society, 2025.
- 665 24 Kim Herzig, Sascha Just, and Andreas Zeller. It’s not a bug, it’s a feature: how misclassification
666 impacts bug prediction. In *2013 35th international conference on software engineering (ICSE)*,
667 pages 392–401. IEEE, 2013.
- 668 25 Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang
669 Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3,
670 2022.
- 671 26 Huihui Huang, Ratnadira Widayarsi, Ting Zhang, Ivana Clairine Irsan, Jieke Shi, Han Wei
672 Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, et al. Back to the basics:
673 Rethinking issue-commit linking with llm-assisted retrieval. *arXiv preprint arXiv:2507.09199*,
674 2025.
- 675 27 Robert L. Logan IV, Ivana Balažević, Eric Wallace, Fabio Petroni, Sameer Singh, and Sebastian
676 Riedel. Cutting down on prompts and parameters: Simple few-shot learning with language
677 models, 2021. URL: <https://arxiv.org/abs/2106.13353>, arXiv:2106.13353.
- 678 28 Maliheh Izadi. Catiss: An intelligent tool for categorizing issues reports using transformers. In
679 *Proceedings of the 1st international workshop on natural language-based software engineering*,
680 pages 44–47, 2022.
- 681 29 Maliheh Izadi, Kiana Akbari, and Abbas Heydarnoori. Predicting the objective and priority
682 of issue reports in software repositories. *Empirical Software Engineering*, 27(2):50, 2022.
- 683 30 Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Ticket tagger:
684 Machine learning driven issue classification. In *2019 IEEE International Conference on*
685 *Software Maintenance and Evolution (ICSME)*, pages 406–409. IEEE, 2019.
- 686 31 Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. Assertflip: Reproducing bugs
687 via inversion of llm-generated passing tests. *arXiv preprint arXiv:2507.17542*, 2025.
- 688 32 Anil Koyuncu. Exploring fine-grained bug report categorization with large language models
689 and prompt engineering: An empirical study. *ACM Transactions on Software Engineering*
690 *and Methodology*, 2025.
- 691 33 D Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses.
692 *social psychological and personality science*, 8 (4), 355–362, 2017.
- 693 34 Van-Hoang Le and Hongyu Zhang. Log parsing with prompt-based few-shot learning. In *2023*
694 *IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2438–2449.
695 IEEE, 2023.
- 696 35 Zhifang Liao, Dayu He, Zhijie Chen, Xiaoping Fan, Yan Zhang, and Shengzong Liu. Exploring
697 the characteristics of issue-related behaviors in github using visualization techniques. *IEEE*
698 *Access*, 6:24003–24015, 2018.
- 699 36 Jiachang Liu, Dinghan Shen, Yizhe Zhang, William B Dolan, Lawrence Carin, and Weizhu
700 Chen. What makes good in-context examples for gpt-3? In *Proceedings of Deep Learning*

- 701 *Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for*
702 *deep learning architectures*, pages 100–114, 2022.
- 703 37 Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni,
704 and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions*
705 *of the association for computational linguistics*, 12:157–173, 2024.
- 706 38 Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy,
707 Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert
708 pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- 709 39 Huan Ma, Changqing Zhang, Yatao Bian, Lemao Liu, Zhirui Zhang, Peilin Zhao, Shu Zhang,
710 Huazhu Fu, Qinghua Hu, and Bingzhe Wu. Fairness-guided few-shot prompting for large
711 language models, 2023. URL: <https://arxiv.org/abs/2303.13217>, arXiv:2303.13217.
- 712 40 Nitish Pandey, Debarshi Kumar Sanyal, Abir Hudait, and Amitava Sen. Automated clas-
713 sification of software issue reports using machine learning techniques: an empirical study.
714 *Innovations in Systems and Software Engineering*, 13(4):279–297, 2017.
- 715 41 Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-
716 networks. *arXiv preprint arXiv:1908.10084*, 2019.
- 717 42 Fabio Santos, Joseph Vargovich, Bianca Trinkenreich, Italo Santos, Jacob Penney, Ricardo
718 Britto, João Felipe Pimentel, Igor Wiese, Igor Steinmacher, Anita Sarma, et al. Tag that
719 issue: applying api-domain labels in issue tracking systems. *Empirical Software Engineering*,
720 28(5):116, 2023.
- 721 43 Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’
722 sensitivity to spurious features in prompt design or: How i learned to start worrying about
723 prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.
- 724 44 Alexander Trautsch and Steffen Herbold. Predicting issue types with sebert. In *Proceedings*
725 *of the 1st international workshop on natural language-based software engineering*, pages 37–39,
726 2022.
- 727 45 Joseph Vargovich, Fabio Santos, Jacob Penney, Marco A Gerosa, and Igor Steinmacher.
728 Givemelabeledissues: An open source issue recommendation system. In *2023 IEEE/ACM 20th*
729 *International Conference on Mining Software Repositories (MSR)*, pages 402–406. IEEE, 2023.
- 730 46 Julian Von der Mosel, Alexander Trautsch, and Steffen Herbold. On the validity of pre-trained
731 transformers for natural language processing in the software engineering domain. *IEEE*
732 *Transactions on Software Engineering*, 49(4):1487–1507, 2022.
- 733 47 Jun Wang, Xiaofang Zhang, and Lin Chen. How well do pre-trained contextual language
734 representations recommend labels for github issues? *Knowledge-Based Systems*, 232:107476,
735 2021.
- 736 48 Xincheng Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun
737 Gao. Aegis: An agent-based framework for general bug reproduction from issue descriptions.
738 *arXiv preprint arXiv:2411.18015*, 2024.
- 739 49 Hongrun Wu, Yutao Ma, Zhenglong Xiang, Chen Yang, and Keqing He. A spatial-temporal
740 graph neural network framework for automated software bug triaging. *Knowledge-Based*
741 *Systems*, 241:108308, 2022.
- 742 50 An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan
743 Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint*
744 *arXiv:2412.15115*, 2024.
- 745 51 Guoxin Yu, Lemao Liu, Haiyun Jiang, Shuming Shi, and Xiang Ao. Retrieval-augmented
746 few-shot text classification. In *Findings of the Association for Computational Linguistics:*
747 *EMNLP 2023*, pages 6721–6735, 2023.